

Presentation
on

Design Pattern (Adapter & Strategy)

CSE 4513

Date: 27/01/2026

Team 1

220041102 Rafat

220041122 Mounota

220041106 Zinnia

220041126 Sakline

220041114 Maha

220041140 Fahim

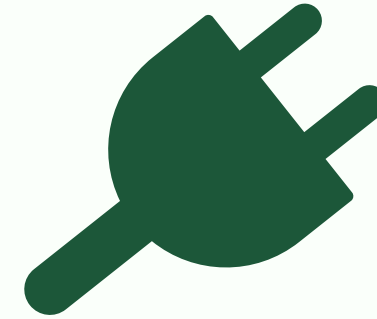
220041118 Rahinur

220041145 Jessan

220041120 Noshin

220041157 Niloy

Adapter Design Pattern



Category: Structural Design Pattern

A design pattern that allows incompatible interfaces to work together by introducing an intermediate adapter.

Why is it a Structural Pattern?

Because it focuses on how classes and objects are composed to form flexible structures.

**Uses Composition
over Inheritance**

**Adjusts incompatible
interfaces**

**Enables seamless
integration preserving
existing code**

Problem It Solves ?

● Incompatible Interfaces

Two classes have incompatible interfaces but compatible functionality.

● Legacy Code Integration

Legacy code must integrate with a new system without modification

● Third-Party Libraries

External libraries whose APIs don't align with the application's interfaces

● Interface Mismatch

Client expects one interface but existing code provides another.

Real-World Examples



File Format Conversion

Software dealing with different file formats (CSV, JSON, XML) uses adapters to convert into a unified format



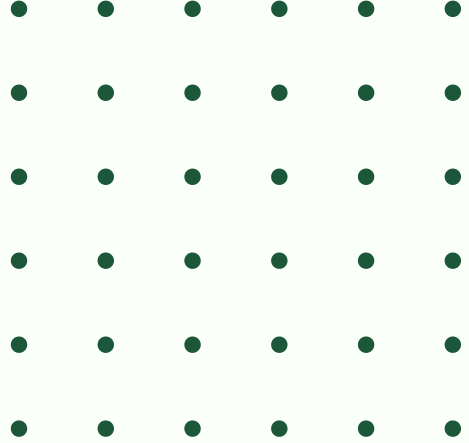
Device Drivers

Operating system device drivers, database connectors, and language converters are practical examples



API Changes

When there's a major change between new vs old APIs, an adapter is used instead of rewriting old code



When to Use the Adapter Pattern?

Communicate Incompatible Systems

To enable interaction between
disparate components.

Reuse Existing Code

To integrate libraries without
extensive rewrites.

Simplify Integration

To facilitate flexible addition of
new components.

Centralize Compatibility

To streamline maintenance and
enhance safety.

Components

01

Target Interface

- Defines the interface expected by the client
- Client interacts only with this interface

03

Adapter

- Implements the Target interface
- Holds an Adaptee instance
- Converts client requests to Adaptee calls

02

Adaptee

- Existing class with required functionality
- Has an incompatible interface

04

Client

- Uses the Target interface
- Remains unaware of Adapter and Adaptee

Structural Flow

Client → Target Interface → Adapter → Adaptee

Coding Example Part 1: Interfaces & Adaptee

1. Target Interface (Printer)

```
class Printer {  
public:  
    virtual void print() = 0;  
};
```

2. Adaptee (LegacyPrinter)

```
class LegacyPrinter {  
public:  
    void printDocument() {  
        std::cout << "Legacy Printer is  
printing a document." << std::endl;  
    }  
};
```

Coding Example Part 2: **Adapter** Implementation

The Adapter class code:

```
class PrinterAdapter : public Printer {  
private:  
    LegacyPrinter legacyPrinter;  
public:  
    void print() override {  
        legacyPrinter.printDocument();  
    }  
};
```


Client Code and Output

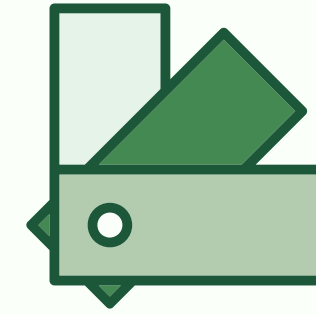
Client Code:

```
void clientCode(Printer& printer) {  
    printer.print();  
}  
  
int main() {  
    PrinterAdapter adapter;  
    clientCode(adapter);  
    return 0;  
}
```

Output:

```
Legacy Printer is printing a document.
```

Strategy Design Pattern



Category: Behavioral Design Pattern

Defines a family of algorithms, encapsulates each one, and allows them to be selected at runtime.

Why is it a Behavioral Pattern?

Because it encapsulates algorithms and enables dynamic behavior selection at runtime.

**Runtime Behavior
Switching without code
modification**

**Algorithm Encapsulation
for Flexibility**

**Interchangeable
Strategies**

Problem It Solves ?

● Multiple Algorithms

Need to support varied algorithms based on runtime conditions.

● Hard-Coded Logic

Direct implementation makes code rigid and difficult to extend.

● Code Duplication

Without structure, logic gets duplicated across the system.

● Limited Flexibility

Adding new algorithms or changing existing ones modifies the main class.

Real-World Examples



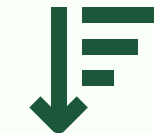
Payment Methods

E-commerce platforms use it for various payment methods (e.g., Credit Card, PayPal), each with distinct processing and verification logic.



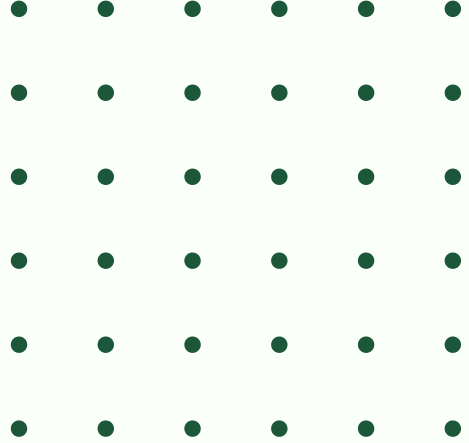
File Compression

Compression utilities offer diverse methods (e.g., ZIP, GZIP) allowing users to select their preferred strategy at runtime.



Sorting Algorithms

Applications sort data using various algorithms (e.g., Merge Sort, Quick Sort) chosen based on data size and performance needs.



When to Use the Strategy Pattern?

Multiple Algorithm Variants

For tasks with multiple algorithms, selectable at runtime.

Avoid Conditional Logic

To eliminate complex if-else or switch statements for algorithm selection.

Easy Algorithm Addition

To simplify adding new algorithms without modifying existing code (Open/Closed Principle).

Runtime Selection

To allow dynamic selection and modification of strategies.

Components

Strategy Interface

- Defines a common interface for all algorithms
- Ensures strategies are interchangeable

Concrete Strategies

- Implement the Strategy interface
- Provide different algorithm implementations

Context

- Maintains a reference to a Strategy
- Delegates the task execution to the selected strategy

Client

- Chooses and sets the strategy at runtime
- Interacts with the Context, not the algorithms

Behavioral Flow

Client → Context → Strategy (selected at runtime)

Coding Example Part 1: Context

```
public class SortingContext {  
    private SortingStrategy sortingStrategy;  
  
    public SortingContext(SortingStrategy sortingStrategy) {  
        this.sortingStrategy = sortingStrategy;  
    }  
  
    public void setSortingStrategy(SortingStrategy sortingStrategy) {  
        this.sortingStrategy = sortingStrategy;  
    }  
  
    public void performSort(int[] array) {  
        sortingStrategy.sort(array);  
    }  
}
```

Coding Example Part 2: Strategy Interface

Strategy Interface (SortingStrategy)

```
public interface SortingStrategy {  
    void sort(int[] array);  
}
```


Coding Example Part 3: Concrete Strategies

BubbleSortStrategy

```
public class
BubbleSortStrategy
implements SortingStrategy {
    @Override
    public void sort(int[]
array) {

        System.out.println("Sorting
using Bubble Sort");
    }
}
```

MergeSortStrategy

```
public class
MergeSortStrategy implements
SortingStrategy {
    @Override
    public void sort(int[]
array) {

        System.out.println("Sorting
using Merge Sort");
    }
}
```

QuickSortStrategy

```
public class
QuickSortStrategy implements
SortingStrategy {
    @Override
    public void sort(int[]
array) {

        System.out.println("Sorting
using Quick Sort");
    }
}
```

Coding Example Part 4: Client Code

```
public class Client {  
    public static void main(String[] args) {  
        SortingContext context = new SortingContext(new BubbleSortStrategy());  
        int[] arr1 = {5, 2, 9, 1};  
        context.performSort(arr1);  
  
        context.setSortingStrategy(new MergeSortStrategy());  
        int[] arr2 = {8, 3, 7, 4};  
        context.performSort(arr2);  
  
        context.setSortingStrategy(new QuickSortStrategy());  
        int[] arr3 = {6, 1, 3, 9};  
        context.performSort(arr3);  
    }  
}
```

Coding Example Part 5: **Output**

Output:

Sorting using Bubble Sort

Sorting using Merge Sort

Sorting using Quick Sort



Thank You